Лабораторные работы №5 и №6

Межъязыковое взаимодействие: интеграция C и Python, FFI, Python pipeline и запуск из C#

Лабораторная работа №5

Реализация функционала на C

Необходимо реализовать на языке **C** вычислительную функцию согласно своему варианту. Функция должна быть доступна для вызова извне, и собрана в динамическую библиотеку:

- Windows: .dll
- Linux: .so

Подключение C-функционала из Python четырьмя способами

Один и тот же функционал на C должен быть использован из Python следующими способами:

1. **Через ctypes**
2. **Через cffi**
3. **Через модуль-расширение на CPython C API**
4. **Через Cython**

Тестирование производительности

Для каждого из четырёх подходов требуется провести серию измерений времени выполнения.

Условия тестирования

Для каждого способа вызова необходимо:

- выполнить **50 запусков теста**;
- в рамках **одного запуска** выполнить **100_000 вызовов** функции;
- во всех 100_000 вызовах использовать **разные входные данные**;
- измерять **общее время одного запуска**;
- по итогам 50 запусков рассчитать:
    - минимальное время;
    - максимальное время;
    - среднее время;
    - медиану;
    - стандартное отклонение.

Важное требование

Для корректности эксперимента:

- сама вычислительная логика должна быть **одинаковой**;
- различаться должен именно **способ доступа к ней из Python**;

Анализ результатов

По итогам тестирования необходимо:

- построить сравнительную таблицу;
- сделать вывод о том,
    - какой способ оказался самым быстрым;
    - какой — самым простым в реализации;
    - какой — наиболее удобным для сопровождения;
    - каковы возможные причины различий в производительности.

Индивидуальная часть по варианту

Реализация C **функции по своему варианту**.

Общие требования к функционалу варианта

1. Функция должна принимать **не менее двух аргументов**.
2. Функция должна возвращать **одно числовое значение** либо структуру с несколькими числовыми значениями.
3. Алгоритм должен быть **детерминированным**.
4. Должна быть предусмотрена обработка некорректных аргументов.
5. Аргументы должны подбираться так, чтобы в 100_000 вызовов они действительно различались.

Варианты заданий

Ниже приведён примерный набор вариантов. Преподаватель может выдать их как есть либо расширить список.

Вариант 1

Реализовать на C функцию вычисления значения полинома:

$$P(x) = a_0 + a_1 x + a_2 x_2 + \cdots + a_n x_n$$

для переданного массива коэффициентов и значения x.

Вариант 2

Реализовать на C функцию вычисления скалярного произведения двух векторов фиксированной длины.

Вариант 3

Реализовать на C функцию вычисления евклидова расстояния между двумя векторами.

Вариант 4

Реализовать на C функцию поиска максимального элемента массива и его индекса.

Вариант 5

Реализовать на C функцию вычисления среднего, дисперсии и стандартного отклонения массива чисел.

Вариант 6

Реализовать на C функцию вычисления значения рекуррентной последовательности с итерационным алгоритмом.

Вариант 7

Реализовать на C функцию подсчёта количества простых чисел на отрезке [L, R].

Вариант 8

Реализовать на C функцию вычисления суммы элементов матрицы, удовлетворяющих заданному условию.

Вариант 9

Реализовать на C функцию умножения двух квадратных матриц малого размера.

Вариант 10

Реализовать на C функцию сглаживания массива методом скользящего среднего.

Вариант 11

Реализовать на C функцию вычисления косинусного сходства двух векторов.

Вариант 12

Реализовать на C функцию подсчёта количества элементов массива, попадающих в заданный диапазон.

Вариант 13

Реализовать на C функцию вычисления значения логистической функции для линейной комбинации признаков:

$$\sigma(w_1x_1 + w_2x_2 + \cdots + w_nx_n + b)$$

Вариант 14

Реализовать на C функцию нормализации массива по Min-Max.

Вариант 15

Реализовать на C функцию нахождения суммы попарных модулей разностей двух массивов.

Требования к реализации части C

1. Исходный код должен быть разделён минимум на:
    - заголовочный файл .h;

    - файл реализации .c.
2. Должна быть предусмотрена возможность сборки в динамическую библиотеку.
3. Все экспортируемые функции должны иметь понятный и единый интерфейс.
4. Следует избегать скрытых глобальных состояний.
5. Если функция использует массивы, необходимо явно передавать:
    - указатель на массив;
    - длину массива.
6. Если возвращается несколько значений, рекомендуется:
    - либо использовать структуру,
    - либо передавать выходные указатели как параметры.

Требования к четырём способам вызова из Python

1. ctypes

Необходимо:

- загрузить динамическую библиотеку;
- указать типы аргументов;
- указать тип возвращаемого значения;
- обеспечить корректную передачу массивов, если они используются.

2. cffi

Необходимо:

- описать C-интерфейс через cdef;
- подключить библиотеку;
- вызвать тот же функционал;
- обеспечить корректное преобразование аргументов.

3. CPython C API

Необходимо:

- создать Python-модуль-расширение на C;
- реализовать функцию-обёртку;
- корректно преобразовывать Python-аргументы в C-значения;
- возвращать результат обратно в виде Python-объекта.

4. Cython

Необходимо:

- создать .pyx-модуль;
- подключить C-функцию через заголовочный файл;
- реализовать Python-обёртку;
- собрать модуль и вызвать его из Python.

Требования к тестированию производительности

Подготовка тестовых данных

Для измерений необходимо заранее сгенерировать набор входных данных, содержащий **100_000 различных наборов аргументов**.

Примеры допустимого подхода

- если функция принимает два числа, то можно заранее сформировать список из 100_000 пар;
- если функция принимает массив, можно заранее подготовить 100_000 массивов или 100_000 индексов/окон в большом массиве;
- если функция принимает матрицы, можно подготовить фиксированное количество шаблонов с модифицируемыми значениями.

Организация измерения

Для каждого подхода:

- выполняется предварительный «разогрев» без учёта в статистике;
- затем выполняется 50 замеров;
- в каждом замере производится 100_000 вызовов функции;
- измерение проводится одной и той же функцией таймера, например:
    - time.perf_counter().

Что нужно сохранить

Для каждого из 50 замеров требуется сохранить:

- номер замера;
- общее время;
- при желании — среднее время одного вызова.

Итоговая таблица

В отчёте должна присутствовать таблица вида:

Подход	**Min**	**Max**	**Mean**	**Median**	**Std**
ctypes					
cffi					

Подход	Min	Max	Mean	Median	Std
CPython C API					
Cython					

Графическое представление

Необходимо построить хотя бы один график:

- столбчатая диаграмма по среднему времени;
  или
- boxplot по всем 50 измерениям.

Лабораторная работа № 6

Python pipeline

После выполнения основной части студент должен создать **Python-скрипт**, реализующий прикладной pipeline обработки данных и предсказания модели.

Требования к pipeline

Pipeline должен включать следующие этапы:

1. **Получение входных данных**
   - из аргументов командной строки;
   - либо из JSON-строки;
   - либо из файла.
2. **Преобразование входных данных**
   - проверка корректности;
   - приведение типов;
   - формирование массива признаков.
3. **Предобработка**
   - нормализация, стандартизация, кодирование или иное преобразование в зависимости от модели.
4. **Загрузка модели**
   - допускается использование простой заранее обученной модели;
   - допустимые варианты:
     - scikit-learn;
     - своя простая модель;
     - линейная/логистическая регрессия;
     - дерево решений;

        - k-NN.

5. **Получение предсказания**
    - вызов predict() или аналогичной функции.
6. **Возврат результата**
    - в стандартный вывод;
    - желательно в формате JSON.

Минимальные требования к модели

Допускается использовать простую модель, например:

- классификация по небольшому набору признаков;
- регрессия по 2–5 признакам;
- модель, обученная на открытом небольшом наборе данных;
- модель, обученная самим студентом на искусственно сгенерированных данных.

Обязательное условие

Модель должна быть реально загружаемой и реально используемой в скрипте.
Недопустимы заглушки вида:

- if x > 10: return 1 else return 0

если это не оформлено как явно собственная математическая модель и не согласовано преподавателем.

программа на C#

Необходимо разработать простое приложение на **C#**, которое:

1. получает входные значения от пользователя;
2. формирует аргументы для запуска Python-скрипта;
3. запускает Python-скрипт как внешний процесс;
4. получает результат работы скрипта;
5. отображает его пользователю.

Минимальные требования к программе на C#

Допускается:

- консольное приложение;
- простое оконное приложение.

Программа должна уметь:

- считывать значения с клавиатуры или из формы;
- вызывать Python-скрипт через Process;
- передавать аргументы командной строки;
- получать stdout;
- обрабатывать возможные ошибки запуска.

Желательно

Если Python-скрипт возвращает JSON, программа на C# должна корректно его вывести или распарсить.

Рекомендуемая структура проекта

Пример структуры каталогов:

```
lab/
├─ c_core/
│  ├─ mylib.h
│  ├─ mylib.c
│  └─ build scripts
├─ python_ctypes/
│  └─ test_ctypes.py
├─ python_cffi/
│  └─ test_cffi.py
├─ python_capi/
│  ├─ module.c
│  └─ setup.py / pyproject.toml
├─ python_cython/
│  ├─ wrapper.pyx
│  ├─ mylib.h
│  └─ setup.py / pyproject.toml
├─ benchmark/
│  ├─ benchmark.py
│  ├─ data_generator.py
│  └─ results.csv
├─ ml_pipeline/
```

```
│  ├─ train_model.py
│  ├─ model.pkl
│  └─ predict.py
├─ csharp_client/
│  └─ project files
└─ report/
   └─ report.docx / pdf
```

Что должно быть продемонстрировано на защите

На защите студент должен показать:

1. исходную реализацию на C;
2. сборку библиотеки;
3. работу вызова через ctypes;
4. работу вызова через cffi;
5. работу модуля через CPython C API;
6. работу через Cython;
7. запуск benchmark-скрипта;
8. таблицу и график результатов;
9. работу Python pipeline;
10. запуск Python pipeline из C#-приложения.

Ограничения и важные замечания

1. Нельзя сравнивать разные алгоритмы под видом разных способов вызова.
2. Нельзя использовать разные входные данные для разных подходов.
3. Нельзя исключать часть накладных расходов выборочно только для одного способа.
4. Недопустимо подменять вызов C-функции реализацией на чистом Python.
5. Если используется случайная генерация тестовых данных, необходимо зафиксировать seed.
6. Все результаты должны быть воспроизводимыми.